

Providing load flexibility by reshaping power profiles of large language model workloads

Yi Wang^{id}, Qinglai Guo^{id}*, Min Chen

Department of Electrical Engineering, Tsinghua University, Shuangqing Road 30, Beijing, 100084, China

ARTICLE INFO

Keywords:

Large language model
Load flexibility
Frequency scaling
Energy efficiency
Renewable energy

ABSTRACT

The emergence of large language models (LLM) has driven a significant increase of AI workload in data center power demand. Renewable-powered solutions to decarbonizing LLM workload and reducing electricity costs are faced with the combined volatility of stochastic user requests and renewable energy. The key to removing the barriers in sustainable AI development lies in the adjustable capability of LLM power profiles. Therefore, this paper focuses on exploring the potential load flexibility of LLM workload and proposes a coordinated scheduling framework, notably, without computing performance degradation. Driven by the existence of the energy-optimal core frequency for graphics processing units (GPU), the energy-performance decoupling phenomenon is discovered and proved, where collaborative scaling in GPU quantity and frequency can change power but not computing performance. Motivated by this, the framework slows down the fine-tuning cluster and utilizes idle GPU resources from the inference cluster to maintain the computing performance of fine-tuning tasks. Consequently, the power consumption of the total cluster is reduced, which provides a fresh source of load flexibility. Furthermore, the framework employs dynamic frequency scaling to more flexibly modify the power profile of the expanded fine-tuning cluster. The computing performance is particularly guaranteed through temporal coupling constraints. In a simulated study supported by real-world data, the results prove a 6.8% power-saving ability and 11.3% cost-saving gains on average.

1. Introduction

The proliferation of generative artificial intelligence (Gen-AI) has positioned data centers as a rapidly growing energy sector, necessitating urgent sustainable solutions [1]. Training an advanced large language model (LLM) like the open-sourced Llama3 70B costs 4.48 million kWh [2], while AI-enhanced Google searches consume 30× more energy per query than standard ones [3]. Consequently, the combined effects of energy intensity and rapid growth suggest AI could drive a 160% surge in data center power demand [4], potentially constituting half of their total consumption within this decade [5]. While global impacts appear moderate at scale, Gen-AI systems may pose stress to regional power grids [6] and even specific national grids, with projections indicating 28% of Ireland's total electricity demand being allocated to data centers by 2030 [7]. These developments underscore that establishing decarbonization pathways for Gen-AI systems holds equal, if not greater, urgency compared to traditional industries, particularly given the sector's emerging transformative potential across multiple domains.

A primary decarbonization pathway involves transitioning power resources for Gen-AI infrastructure from fossil fuels to renewable energy. Hydroelectricity, along with nuclear power, can be excellent

baseload resources, due to their low carbon profiles and high reliability. Many regions like Quebec [8], Norway [9], and parts of the U.S. Pacific Northwest already rely on hydropower for data center operations. For areas with limited hydro or nuclear resources, the energy supply for AI systems necessitates the integration of other renewables. In China, newly-built data centers are required to consume at least 80% green power by 2025 for the carbon neutral goal. In this context, wind and solar are competitive options, given its coverage of 86% newly installed capacity in 2024 and the supportive policy for direct and exclusive physical supply. Besides, a heterogeneous mix of different generation technologies, which incurs different degrees of fixed and variable costs, can be more cost-effective than a single technology [10]. Overall, the widespread availability and low construction costs render solar photovoltaics and wind farms crucial to scalable, sustainable AI energy supply.

However, the heightened volatility stemming from user demand, solar power generation, and wind power output creates substantial barriers to the decarbonization efforts of renewable energy solutions for Gen-AI applications [11]. Similar to renewable energy sources that rely on complicated atmospheric systems, user demand patterns emerge

* Corresponding author.

E-mail address: guoqinglai@tsinghua.edu.cn (Q. Guo).

from the aggregation of vast human behaviors, which are inherently stochastic and unpredictable. While information and communications technology companies have increasingly procured renewable energy through power purchase agreements [12], these contractual arrangements fail to stimulate additional renewable energy production. This is because the fundamental issue of maintaining real-time power balance under increasing uncertainty is merely transferred rather than resolved. Consequently, the actual emission reductions achieved through such measures remain uncertain and may not contribute effectively to long-term decarbonization goals.

The key to breaking the barriers is the load flexibility of the emerging but fast-growing LLM workload. If the power demand of LLM workload can be controllable, it can partly compensate for the volatility of stochastic renewable generation. Fortunately, LLM workload can potentially offer flexible power profiles via intermediate control measures. From the system level, schedulers can determine when to start, stop, or resume LLM tasks, and how many graphic processing units (GPUs) are launched for each task, influencing the total power of GPU clusters. From the hardware level, power of each GPU device can be modified by dynamic voltage and frequency scaling (DVFS) [13].

Therefore, the main objective of this paper is to develop a method of smartly reshaping AI power profiles, especially for LLM workloads. Both single- and multi-interval load flexibility will be explored. The former flexibility can be applied in demand response programs [14–16], whereas the latter can provide the ability to chase dynamic price signals or renewable energy outputs, which can be useful in promoting cost efficiency and reducing carbon emissions. Specifically in the scenario of dedicated baseload supply, the volatility in power profiles due to intermittent inference requests can be relieved for better power matching by exploiting multi-interval load flexibility.

For one, researchers from the energy field have explored load flexibility of data centers for general computation [17], but rarely consider the specialty of deep learning workload, with complex relations between control measures and power, plus side effects of changing power in undermining the computing performance. For another, performance-oriented scheduling schemes [18] for GPU data centers have been developed by researchers on parallel computing, but only a few incorporate energy-aware functionalities. [19] discovers the trade-off between the overall energy consumption and performance optimization, and proposes an online framework to find optimal task- and GPU-level configurations. The authors in [20] design a renewable-aware frequency scaling approach for deep learning clusters, where training task start-stop and GPU core frequency are tuned to maximize renewable energy utilization. An energy-efficient GPU cluster scheduling method for deep neural network training is proposed in [21], which reduces the average job completion time under an energy budget by tuning GPU allocation and clock frequency. Other than training workloads, the authors in [22] coordinate batching and DVFS to maximize energy efficiency for convolutional neural network inference. The impact levels of possible control knobs on power and performance of deep neural network inference are investigated in [23] to help achieve high throughput under fluctuated renewable energy supply. In summary, existing energy-aware scheduling methods for deep learning workload have not paid attention to the coordination between training and inference, and unavoidably cause prolonged completion time of training tasks.

To fill the gap, this paper proposes a coordinated scheduling framework for LLM fine-tuning and inference to exploit potential load flexibility, notably, without computing performance degradation. Contributions are summarized as follows:

- The load flexibility of LLM workload is extracted for the first time, to the best knowledge of the authors. The existence of the energy-optimal GPU frequency is first proved, and the energy-performance decoupling phenomenon is then derived, where joint scaling in GPU frequency and quantity reduces energy but not computing performance. Furthermore, temporal collaboration is considered to enhance power adjustment ranges.

- A coordinated scheduling framework for LLM fine-tuning and inference workload is proposed to exploit single- and multi-interval load flexibility. On an hour-level time scale, the maximum number of spare GPUs in LLM inference is identified and reallocated to LLM fine-tuning, which offers single-interval load flexibility. On a minute-level time scale, sophisticated configurations including GPU frequency for fine-tuning workload are determined, which provides multi-interval load flexibility.
- A case study driven by real-world data proves that employing the load flexibility of LLM workload produces averagely 6.8% energy-saving effects. Moreover, the dynamic scheduling strategy decreases electricity costs by 11.3% under a practical electricity pricing policy.

The organization of this paper is as follows: Section 2 explains the underlying principle for exploiting the load flexibility of LLM fine-tuning workloads. Section 3 introduces the overall scheduling framework. Sections 4 and 5 explicate the detailed methods of exploiting single- and multi-interval load flexibility under the scheduling framework, respectively. Results are analyzed in Section 6, and future applications are discussed in Section 7. Finally, Section 8 concludes the paper.

2. Extract the load flexibility of LLM fine-tuning

One of the trends in LLM applications lies in fine-tuning foundation models for specific scenarios in enormous fields. Normally, the foundation models are built from open-sourced data resources with varying forms and do not require repeated training, at least not as much as the demand driven by uncountable downstream fine-tuning tasks. Besides, the fine-tuned models need to be renewed when new data from users rush in. Therefore, we mainly focus on the power characteristics of LLM fine-tuning tasks.

Most importantly, we discover and prove an interesting phenomenon in LLM fine-tuning workload: more GPUs with slower GPU clock frequencies consume less power than fewer GPUs with higher GPU clock frequencies to finish the same fine-tuning task within the same time. As computing performance does not change but total energy changes, we name the above characteristic the energy-performance decoupling phenomenon. Simply put, this potential is driven by different impact levels of GPU core frequency on power consumption and computing performance. GPU power is proportional to the cube of clock frequency, whereas the time to finish a task is inversely proportional to clock frequency.

In this section, the underlying principle on how to extract the load flexibility of LLM fine-tuning is uncovered, as shown in Fig. 1. The existence of the energy-optimal GPU frequency is first theoretically proven. Based on the proof, the energy-performance decoupling phenomenon is then analyzed in detail, which offers potential downward load flexibility. Furthermore, the cooperation between time intervals is considered, where the maximum downward flexibility is increased with emergent upward flexibility.

2.1. Existence of energy-optimal GPU frequency

For one, power consumption of one GPU can be divided into dynamic and static components. The dynamic part comes from computing and memory operation, whereas the static one includes power consumed by sleepy components and interior support circuitry [24]. Previous studies [25,26] have proven that the two power components are linearly related to V^2f and V , respectively, where V denotes the support voltage and f is the GPU core frequency. Additionally, researchers have found the voltage V has a near-linear relation to the frequency f [27,28], which explicates the power model $P_j(f)$ for the LLM fine-tuning task j :

$$P_j(f) = \alpha_j^P * f^3 + \beta_j^P * f + \gamma_j^P \quad (1)$$

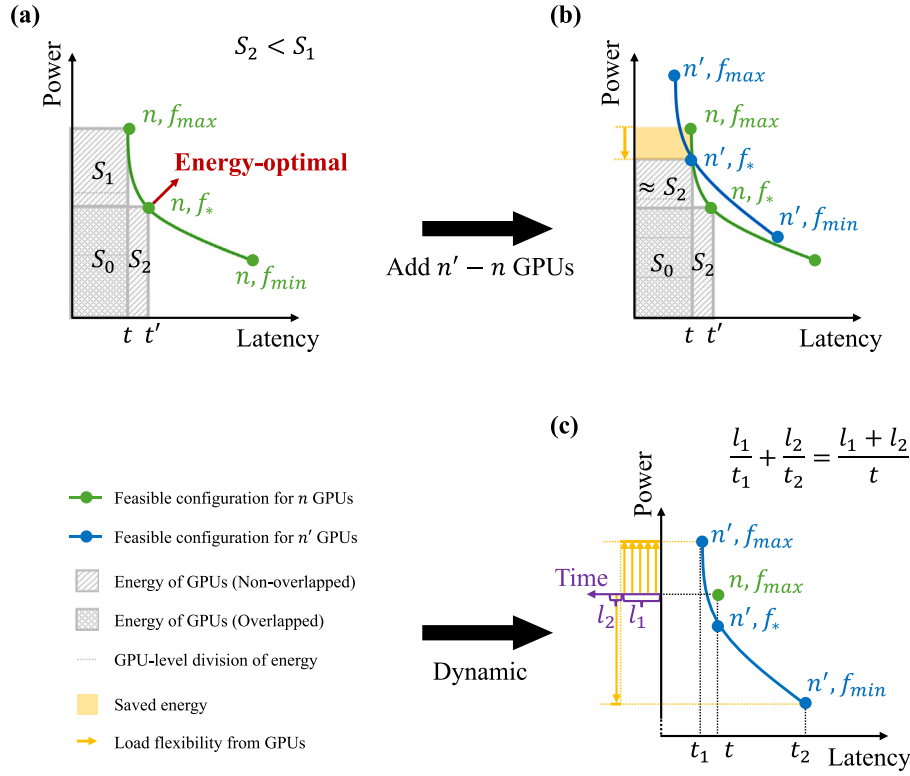


Fig. 1. Illustration for load flexibility of LLM fine-tuning workload. (a), The existence of the most energy-efficient GPU frequency. (b), Downward load flexibility from adding GPUs (static frequency setting). (c), Upward and downward load flexibility through temporal cooperation (dynamic frequency setting).

where $\alpha_j^P, \beta_j^P, \gamma_j^P$ are fittable parameters.

For another, the latency T_j of each model update consists of reading, forward, synchronization, back-propagation, and writing. Here, we follow the modeling in [21]. Firstly, loading models and data into GPUs involves storage IO, which costs the time T_j^{IO} modeled as linearly dependent on the number of GPUs n due to bandwidth competition. Secondly, the time T_j^{grad} spent on computing gradients is inversely proportional to GPU core frequency. Lastly, the time for gradient aggregation T_j^{sync} is linearly related to the number of GPUs, similar to IO modeling. The linear communication overhead assumption is empirically justified as in [21,29]. The underlying reason is the linearly increasing communication volume with respect to the GPU amount under a fixed bandwidth. Here, intra- and inter-node data transfer is assumed to cause equivalent communication latency, which holds true at least for L40s servers [30].

$$T_j = T_j^{IO} + T_j^{grad} + T_j^{sync} \quad (2)$$

$$T_j^{IO} = \alpha_j^{IO} + \beta_j^{IO} n \quad (3)$$

$$T_j^{grad} = \alpha_j^{grad} + \beta_j^{grad} f^{-1} \quad (4)$$

$$T_j^{sync} = \begin{cases} 0 & , n = 1 \\ \alpha_j^{sync} + \beta_j^{sync} n & , n \geq 2 \end{cases} \quad (5)$$

where $\alpha_j^{IO}, \beta_j^{IO}, \alpha_j^{grad}, \beta_j^{grad}, \alpha_j^{sync}, \beta_j^{sync} \geq 0$ are parameters to be fitted for the LLM fine-tuning task j . The latency model $T_j(n, f)$ can be transformed into the simplified version with fewer parameters $\alpha_j^T, \beta_j^T, \gamma_j^T$.

$$T_j(n, f) = \alpha_j^T + \beta_j^T f^{-1} + \gamma_j^T n \quad (6)$$

Combining the power model with the latency model gives the energy model $E_j(n, f) = n P_j(f) T_j(n, f) \propto P_j(f) \hat{T}_j^n(f) = \hat{E}_j^n(f)$, where $\hat{T}_j^n(f)$ and $\hat{E}_j^n(f)$ denote the latency and energy model for a fixed n , respectively.

$$\hat{T}_j^n(f) = \hat{\alpha}_j^T + \hat{\beta}_j^T f^{-1} \quad (7)$$

$$\begin{aligned} \hat{E}_j^n(f) = & \hat{\alpha}_j^T \alpha_j^P f^3 + \beta_j^T \alpha_j^P f^2 + \hat{\alpha}_j^T \beta_j^P f \\ & + (\hat{\alpha}_j^T \gamma_j^P + \beta_j^T \beta_j^P) + \beta_j^T \gamma_j^P f^{-1} \end{aligned} \quad (8)$$

where $\hat{\alpha}_j^T = \alpha_j^T + \gamma_j^T n$.

Proposition 1. For any $j \in [J], n \in \mathcal{N}^+$, there exists a minimum point $\xi \in \mathcal{R}^+$ for $\hat{E}_j^n(f)$ such that $\hat{E}_j^n(\xi) < \hat{E}_j^n(f), \forall f \in \mathcal{R}^+ \setminus \{\xi\}$.

The theoretical proof depends on the convexity of (8) in the field of positive real numbers, as derived in detail in Appendix A.1. Besides, the existence of the most energy-efficient frequency is empirically proven in [21,22] and our experiment, which uncovers that this particular frequency is often smaller than the technically supported maximum frequency. Notably, the most energy-efficient frequency here might not be the theoretical point ξ , as real frequencies are discrete values, which also explains that μ_1 and μ_2 are only chosen to apply the intermediate value theorem, and might not be a configurable value.

2.2. Energy-performance decoupling phenomenon

Consider two clusters that can finish the same fine-tuning task j at the same time: One works under the default maximum GPU frequency f_j^0 with n_j^0 GPUs, whereas the other runs under the most energy-efficient frequency $f_j^1 = \arg\min \hat{E}_j^n < f_j^0$ with expanded $n_j^1 > n_j^0$ GPUs. The energy consumed by n_j^0 GPUs under core frequency f_j^0 and f_j^1 is denoted as e_j^0 and e_j^1 . Strictly speaking, the “same” task here represents that the used data samples for model updates are exactly the same, both in volume and content, and the “cluster” here is particularly limited to a group of GPUs needed for one task.

Definition 1. Computing performance is the averaged time cost of using one data sample to fine-tune an LLM (named per-sample latency) or the reciprocal (named sample throughput). For the j th task, the two

equivalent calculations of the computing performance v_j^{-1} and v_j are:

$$v_j^{-1} = \frac{T_j(n_j, f_j)}{n_j * b} \text{ (per-sample latency)} \quad (9)$$

$$v_j = \frac{n_j * b}{T_j(n_j, f_j)} \text{ (sample throughput)} \quad (10)$$

where b is the local batch size, namely, the number of samples executed in one GPU during one update.

Given the equivalence in the computing performance of both clusters, we can prove the following key finding. The detailed derivation is shown in [Appendix A.2](#).

Proposition 2. *Cluster-1 with (f_j^1, n_j^1) configuration costs less power than Cluster-0 with (f_j^0, n_j^0) configuration, as long as $\gamma_j^T < \frac{(1-E)(\alpha_j^T + \beta_j^T / f_j^1)}{|KE-1|n_j^0}$ where*

$$K = \frac{\alpha_j^T + \beta_j^T / f_j^1}{\alpha_j^T + \beta_j^T / f_j^0} \text{ and } E = \frac{e_j^1}{e_j^0}.$$

Consequently, the energy-saving effect after GPU expansion can be regarded as a fresh resource of downward reserve, as [Fig. 1\(b\)](#) demonstrates. The incremental communication latency per GPU added, denoted as γ_j^T , should be upper limited to accomplish this. Notably, the analysis here takes the most energy-efficient frequency as an example, which can be replaced by any frequency that achieves more energy efficiency than the default maximum one. Merely the corresponding energy-saving capacity varies between the frequency selections.

Additionally, the downward flexibility can be further increased through temporal cooperation in dynamic scenarios, as shown in [Fig. 1\(c\)](#). The expanded cluster can achieve faster computing speed if GPU core frequency is tuned to exceed the energy-optimal one, which can offset the computing performance undermining caused by slowing down GPUs in other intervals.

3. Establish the coordinative scheduling framework

LLM inference workload exhibits strong intraday fluctuations [31], causing partial GPUs to be idle yet still consume power as redundancy (at least 11% of total power for a GV100 GPU [24]). Recognizing this inefficiency, and based on previous analysis, we identify an opportunity: these temporally idle GPUs can be dynamically repurposed for LLM fine-tuning tasks, thereby unlocking valuable load flexibility within the computing cluster.

Therefore, a novel coordinative scheduling framework is proposed, where LLM fine-tuning and inference workloads share a GPU resource pool. The setting of unified resource management is both technically and practically reasonable. Technically, contemporary GPU architectures are well-suited to execute both training (fine-tuning) and inference computations. Practically, consolidating GPU resources for these distinct but hardware-compatible workloads can enhance operational flexibility and improve overall resource utilization efficiency.

The proposed framework is illustrated in [Fig. 2](#). Within each scheduling epoch, the framework first employs forecasting techniques to predict upcoming inference request volumes. This prediction enables the identification of GPUs anticipated to be idle during the current epoch and thus available for reallocation. Concurrently, based on offline performance and power profiling data for various model and hardware configurations, task-level GPU allocation strategies for pending fine-tuning workloads are determined.

From the perspective of fine-tuning scheduling, idle GPUs are then reallocated to fine-tuning tasks, which can make the cluster expand, slow down, and consume less power. If the electric tariff or renewable generation is given for reference, the minute-level scaling strategy of GPU frequency is further worked out to provide desirable dynamic power profiles. From the perspective of inference scheduling, nothing has changed except for an accessible GPU pool of variable size, changing according to possible request volume. The remaining GPUs

reserved for second-level batched serving are sufficient to satisfy peak demand in the period. Besides, in case inference requests suddenly rise unexpectedly, the whole cluster can quickly return to the prior efficient working condition. This also explains why we reallocate idle GPUs to fine-tuning tasks rather than directly scheduling the expanded cluster.

As designed to be compatible with existing cluster management systems, the implementation of the framework can rely on established mechanisms for normal functionalities including GPU reallocation and frequency scaling, along with fault tolerance such as job preemption and GPU failures. GPU reallocation can be realized using checkpoint-restart techniques within minutes [29], whereas frequency scaling can be achieved through NVIDIA Management Library [32] at the millisecond level [33]. Besides, the current implementation applies to multi-tenant scenarios prevalent in cloud services, which facilitates resource sharing between tenants at an inter-GPU level.

In essence, this coordinative framework enables the exploitation of both single-interval and multi-interval load flexibility of LLM workloads. The former is achieved by integrating Step 2 (GPU Status Identification) and Step 3 (Idle GPU Reallocation) into existing scheduling methods (highlighted in blue). The latter is realized by further incorporating Step 4 (Frequency-Load Characteristics Aggregation) and Step 5 (Dynamic GPU Frequency Scaling).

4. Exploit single-interval load flexibility

In this section, an energy-efficient GPU reallocation method is proposed to avoid useless power waste without undermining the overall computing performance of LLM workload. The method also uncovers the downward flexibility that GPU clusters can offer to power grids or deal with inner-level power shortage emergencies. Firstly, available idle GPUs in the original resource pool for inference are determined based on request forecasts. Using the retained GPUs, LLM inference workload can always attain the best energy efficiency and satisfy consumer requests within specified latency during one reallocation period. Secondly, idle GPUs are reallocated to LLM fine-tuning workload in a greedy way to maximize the potential energy savings from expanding and down-clocking. The enlarged GPU cluster for executing fine-tuning tasks will be configured with the decreased GPU core frequencies that provide the same computing performance.

4.1. GPU status identification

Assume there are M sets of completion time and energy consumption $(t_1, e_1), \dots, (t_M, e_M)$ for executing an inference job, obtained off-line under the configuration combos of batch size and GPU frequency $(b_1, f_1), \dots, (b_M, f_M)$. The impact of GPU quantity n on performance and energy when finishing l requests can be further formulated as set out in [Appendix B.1](#). Combining (B.1) and (B.2), the optimization model (B.4) minimizing total energy consumption for running L_m requests within the latency threshold T_0 can be transformed into:

$$\min_{n, x} L_m \sum_{i \in [M]} \left(\frac{e_i}{b_i} x_i \right) \quad (11a)$$

$$s.t. \sum_{i \in [M]} \left(\frac{t_i}{b_i} x_i \right) \leq \frac{T_0}{L_m} n \quad (11b)$$

$$1 \leq n \leq L_m \sum_{i \in [M]} \left(\frac{1}{b_i} x_i \right) + 1, n \in \mathcal{N} \quad (11c)$$

$$\sum_i x_i = 1, x_i \in \mathcal{B}, \forall i \quad (11d)$$

where the integer variable x_i stands for whether to select the i th configuration combo. Here, the floor function in (B.1) is neglected in that l is usually much larger than bn for large-scale LLM clusters. The model (11) indicates a direct iterative solution method where the most energy-efficient GPU configuration (b^*, f^*) is first identified, and the optimal reserved GPU amount n^* is thus determined by dividing peak

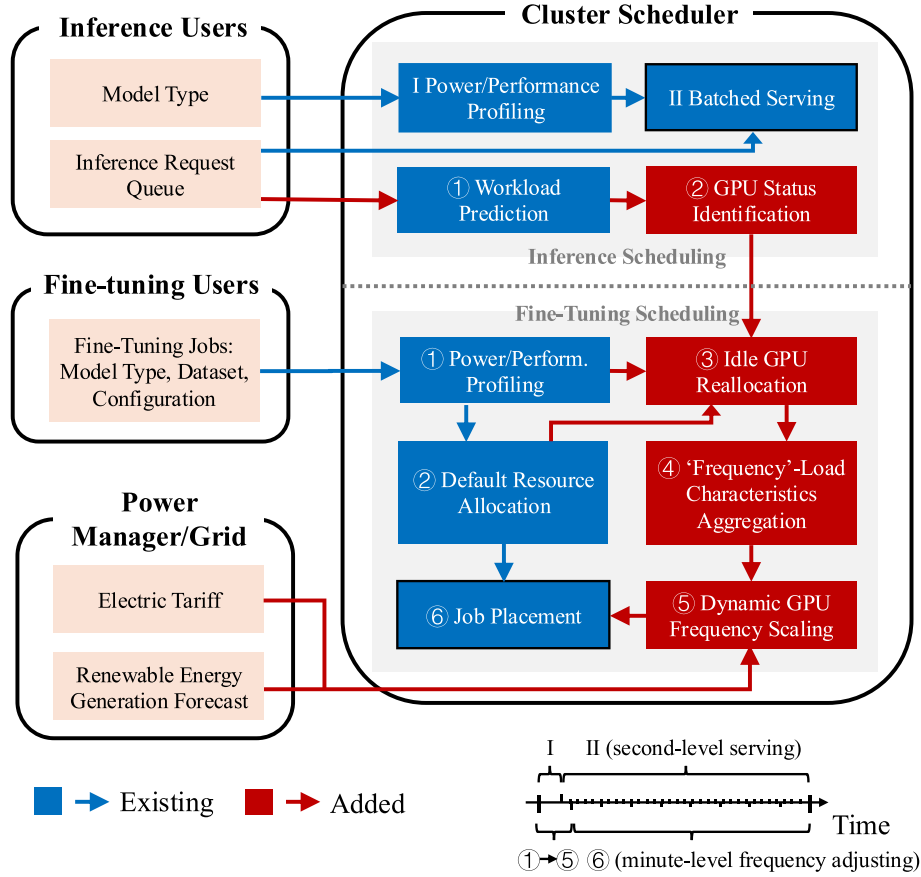


Fig. 2. A collaborative scheduling framework for LLM workload without computing performance degradation.

demand L_m by b^* if satisfying (11b); otherwise, the second energy-efficient combo is selected, and the iteration stops until the latency constraint is met. Then, the number of available idle GPUs N_{id} is the default GPU quantity for inference N_{in}^0 minus the reserved GPU quantity n^* .

4.2. Idle GPU reallocation

Before idle GPU reallocation, the default task-level configuration (n_j^0, f_j^0) is known for each task j , determined by the user when uploading the task, or by the cluster scheduler. Similar to inference, we can firstly obtain power and performance profiling results for fine-tuning, and then determine M_j setting combinations $(n_j^m, f_j^m), m \in [M_j]$ producing the same per-sample latency as the default setting (n_j^0, f_j^0) and consuming probably different power p_j^m . Without loss of generality, GPU quantity n_j^m is in ascending order with m .

When reallocating idle GPUs, it is desirable that the free resources go to where they can bring maximum energy-saving profits. Therefore, an iterative and greedy algorithm can be developed to quickly obtain the optimal reallocation solution Y^* . Starting from the default allocation scheme, one idle GPU is reallocated to the task that saves the most energy when adding one GPU and down-clocking until all the idle GPUs have been placed. The complete algorithm can be seen in Alg. 1, where the available GPU amount for fine-tune workload N_{ft} equals the default resources for fine-tuning N_{ft}^0 plus idle ones N_{id} .

5. Exploit multi-interval load flexibility

Employing the method in Section 4 can output an expanded, down-clocking GPU cluster for LLM fine-tune workload that holds the computing performance unchanged. This section further explores the potential

Algorithm 1 Obtain optimal GPU allocation Y^*

Require: $n_j^m, p_j^m, \forall m \in [M_j], \forall j \in [J], N_{ft}$
Ensure: $Y^* = \arg\max_Y \text{GetPower}(Y), N_{ft} = \text{GetNum}(Y)$

- 1: $y_j^m \leftarrow 0, \forall j, m$
- 2: $y_j^1 \leftarrow 1, \forall j$
- 3: **while** $\text{GetNum}(Y) \leq N_{ft}$ **do**
- 4: $Y_{temp}(j) \leftarrow \text{AddOneGpu}(Y, j)$
- 5: $P_{temp}(j) \leftarrow \text{GetPower}(Y_{temp}(j))$
- 6: **if** $\min_j P_{temp}(j) < \text{GetPower}(Y)$ **then**
- 7: $j_{add} \leftarrow \arg\min_j P_{temp}(j)$
- 8: $Y \leftarrow \text{AddOneGpu}(Y, j_{add})$
- 9: **else**
- 10: **break**
- 11: **end if**
- 12: **end while**
- 13: **Function** $\text{GetNum}(Y)$
- 14: **return** $\sum_{j,m} y_j^m n_j^m$
- 15: **Function** $\text{GetPower}(Y)$
- 16: **return** $\sum_{j,m} y_j^m p_j^m$
- 17: **Function** $\text{AddOneGpu}(Y, j)$
- 18: **while** $k \leq M_j - 1$ **do**
- 19: **if** $y_j^k == 1$ **then**
- 20: $y_j^k \leftarrow 0, y_j^{k+1} \leftarrow 1$
- 21: **end if**
- 22: $k \leftarrow k + 1$
- 23: **end while**
- 24: **return** Y

dynamic flexibility LLM workload can exploit, which is reflected in the ability of chasing dynamic signals and accordingly reshaping its power series. The signals can be future renewable energy generation, grid-level carbon intensity, electricity costs, and so on.

Firstly, we build the power characteristics of the whole cluster with respect to GPU core frequency by aggregation. Then, a dynamic scheduling method is established for a potential practical application based on the aggregated ‘frequency’-load characteristic. The method provides a reshaped power series of the cluster that minimizes total power purchase costs and ensures average computing performance.

5.1. ‘Frequency’-load characteristics aggregation

To dynamically adjust the power of LLM fine-tuning workload, the ‘frequency’-load characteristic should be first built. Notably, the ‘frequency’ here denotes the GPU core frequency, which is the major control knob, instead of the grid frequency. Besides, accompanied by changing GPU core frequency, the impacts on the computing performance need to be considered.

The most direct and accurate way to characterize the feasible power curve concerning GPU core frequency is to assign K_j integer variables for each task j where K_j denotes the number of viable frequency options. Each decision variable reflects whether or not to set the core frequency of GPUs carrying task j to f_j^k . However, the dimension of the optimization problem rises linearly with the number of tasks, which leads to scalability problems.

Therefore, numerous tasks are considered as a whole by aggregation to adjust the total power. The optimal static configuration (n_j^*, f_j^*) obtained in Section 4.2 is labeled as the initial state. Then, the feasible operation states including the global sample throughput V_u and global power P_u can be obtained by changing the clock frequency of all fine-tuning tasks simultaneously by δf_j at a time. The frequency interval δf_j is an integer multiple of the minimum adjustable interval decided by hardware design.

$$V_u = \frac{1}{J} \sum_j \frac{n_j^* * b}{T_j(n_j^*, \min(f_j^* + u * \delta f_j, f_j^{K_j}))}, \forall u \in \mathcal{N} \quad (12)$$

$$V_{-u} = \frac{1}{J} \sum_j \frac{n_j^* * b}{T_j(n_j^*, \max(f_j^* - u * \delta f_j, f_j^{K_j}))}, \forall u \in \mathcal{N} \quad (13)$$

$$P_u = \sum_j n_j^* P_j(\min(f_j^* + u * \delta f_j, f_j^{K_j})), \forall u \in \mathcal{N} \quad (14)$$

$$P_{-u} = \sum_j n_j^* P_j(\max(f_j^* - u * \delta f_j, f_j^{K_j})), \forall u \in \mathcal{N} \quad (15)$$

where u takes the threshold when the GPU frequency of all the tasks has reached the rate value (f_j^0 or $f_j^{K_j}$). Here, the throughput version of computing performance is used instead to make the constraint of equivalent performance easier to represent and understand in the dynamic scenario with fixed intervals.

5.2. Dynamic GPU frequency scaling

The method with static frequency setting in Section 4.2 cannot respond to dynamic signals, such as renewable energy outputs. Assume an extreme case where wind power resources turn sufficient enough to cover the existing electric load; there is no reason not to accelerate the LLM training process by boosting GPU core frequency. Then, the problem arises: which tasks to accelerate and how fast GPUs should run. The principle of contrary situations is similar but with the opposite direction of frequency scaling. Besides, the computing performance should not be undermined or at least controllable. This characteristic is due to the urgent demand for downstream applications with potential profits after introducing AI.

Here, we take a potential application as an illustrative example to show how to exploit the dynamic flexibility. The cluster prioritizes

the consumption of self-built photovoltaic systems and wind farms, and purchases power from the grid for the imbalanced part. Then, the LLM fine-tuning dynamic scheduling strategy can be retrieved by solving the following mixed-integer optimization problem (16). The objective (16a) includes volumetric charges, capacity charges, and fixed charges as three fundamental structures in grid tariffs. C_α , C_β and C_γ are corresponding tariff coefficients, respectively.

$$\min_{z_u^t, P_s^t, P_w^t, P_r^t} \sum_{t \in [N_T]} C_\alpha^t * P_r^t + C_\beta * P_m + C_\gamma \quad (16a)$$

$$s.t. \sum_u z_u^t P_u^t + P_v^t = P_r^t + P_w^t + P_s^t, \forall t \in [N_T] \quad (16b)$$

$$\underline{P_s^t} \leq P_s^t \leq \overline{P_s^t}, \forall t \in [N_T] \quad (16c)$$

$$\underline{P_w^t} \leq P_w^t \leq \overline{P_w^t}, \forall t \in [N_T] \quad (16d)$$

$$\sum_u z_u^t = 1, z_u^t \in \mathcal{B}, \forall u, t \in [N_T] \quad (16e)$$

$$P_m \geq P_r^t, \forall t \in [N_T] \quad (16f)$$

$$\frac{1}{N_T} \sum_{t \in [N_T]} \sum_u z_u^t V_u \geq \eta V_0 \quad (16g)$$

where P_v^t , P_w^t , P_s^t and P_r^t represent the power of inference workload, wind power, solar power, and purchased power from the grid at the time interval t , respectively. P_m is the maximum purchased power among N_T steps in one scheduling period. z_u^t determines whether the GPU cluster works under the state u at the time interval t . For each time interval t , only one feasible state can be selected. Besides, solar power generation, as well as wind power, is bounded between $\underline{P_{s(w)}^t}$ and $\overline{P_{s(w)}^t}$ due to atmospheric conditions and machinery limits. η denotes the global throughput ratio, setting which to 1 means that the solution achieves an average global sample throughput equal to V_0 under the initial configuration.

6. Case studies

In this section, the profits from exploiting single-interval and multi-interval load flexibility under the coordinative scheduling framework are evaluated by a one-week simulation experiment driven by real-world data. For one, power consumption of the GPU cluster is compared before and after GPU reallocation and down-clocking under varying idle resources. For another, the cost-saving profits of reshaping power series by dynamic frequency scaling are studied in an illustrative application case, where the default cluster and the expanded one are separately tested and compared.

6.1. Experimental setup

Hardware and software configurations for profiling, simulation settings based on real-world and benchmark methods to be experimented are first clarified.

6.1.1. Profiling configuration

Power and performance profiling results for both inference workload and fine-tuning workload are obtained using the state-of-the-art Qwen2.5-32B-Instruct model [34], and experimented on NVIDIA A800 GPUs, which serve as an alternative to the commonly used A100 GPUs [35]. Query and control of GPU working states such as GPU core frequency are realized by NVIDIA Management Library [32]. Specifically for fine-tuning tasks, memory limitations prompted the use of bfloat16 precision for model loading and the application of Low-Rank Adaptation techniques, yielding roughly 10% trainable parameters. Data parallelism, facilitated by the Hugging Face Transformers library, is employed to implement distributed training.

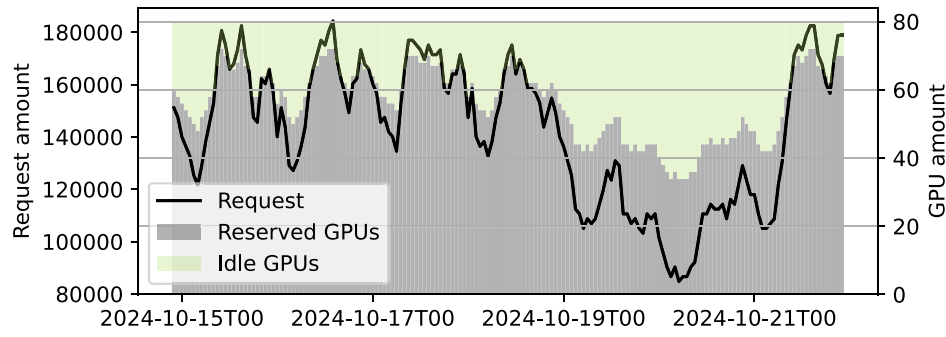


Fig. 3. Request amount of LLM inference workload and available idle GPUs.

6.1.2. Simulation settings

The Google Trends platform offers access to hourly interest levels regarding any topic, for example, ChatGPT specified here. The interest level series is then used and scaled to simulate the request amount due to the strong correlation between the two terms. The minute-level inference requests, solar power, and wind power come from AzureLLMInferenceDataset [31], ARENA Solar Power Ensemble Forecaster Project [36], and the WIND dataset [37], respectively.

The GPU cluster is by default composed of 80 and 100 GPUs working at 1230 MHz core frequency for inference and fine-tuning workloads, respectively. Given the data sources, basic settings and the method in Section 4.1, the hourly available idle GPUs can be obtained, which are a critical input for validating the profit from exploiting load flexibility afterward. As can be seen from Fig. 3, available idle GPUs for reallocation range between zero and even half of the GPU resource pool for inference, which trend resembles that of requests and proves the significant potential for resource utilization promotion. The offline profiling results, as the information basis for GPU status identification, can be seen in Appendix B.3.

The computing cluster is supplied with approximately 50% penetration of renewable energy and purchases power from the grid. The pricing framework, including the values, adheres to China's real-world energy policies: the volumetric charge coefficient is set to 0.1357 CNY/kWh (adjustable within 30%–190% of the base rate), while the capacity charge coefficient C_β is 51.2 CNY/kW·month, and scaled to 0.071 CNY/kWh to match the scheduling interval. This conversion results in a lower estimated monthly electricity bill, but makes sense when optimizing operation.

6.1.3. Benchmark methods

To evaluate the proposed framework, three management strategies are simulated: Static Allocation Static Frequency (SASF), Dynamic Allocation Static Frequency (DASF), and Dynamic Allocation Dynamic Frequency (DADF).

- SASF: This strategy serves as the baseline. It maintains the default allocation of GPUs for inference and fine-tuning and leaves spare resources idle.
- DASF: This strategy enhances the baseline by employing the GPU reallocation method detailed in Section 4. The computing performance, represented by per-sample latency, stays unchanged.
- DADF: Building upon DASF, this strategy additionally incorporates dynamic frequency scaling technique described in Section 5. The global throughput ratio η in (16g) is set to 1 in DADF, ensuring a consistent level of the averaged global sample throughput.

In our case study, both DASF and DADF are benchmarked against the SASF baseline to quantify their improvements in energy savings and cost-effectiveness respectively.

6.2. Energy-saving profits from single-interval load flexibility

The energy-saving potential is reflected in Fig. 4 when the per-sample latency is around 1.7 and 2.5 s, as the power of fewer fast-clocking GPUs exceeds that of more slow-clocking GPUs. The saved energy by GPU quantity expansion from 3 to 4 is especially demonstrated by the red-colored one-way arrow in the bottom sub-figure as an example. Notably, the energy saved deviates from the optimal value, as GPUs after reallocation do not work under the energy-optimal core frequency in this case. The deviation in energy saving is depicted by a two-way red-colored arrow in the top sub-figure. Overall, the result provides empirical proof of the phenomenon theoretically analyzed in Section 2. Using the configurations that maintain the same per-sample latency, we further analyze possible energy-saving capabilities with available GPUs obtained in Section 6.1.2.

The total energy consumption of the GPU cluster can be averagely reduced by 6.8% within one week on an hourly basis, as shown in Fig. 5. Here, the cluster is assumed to carry 20 fine-tuning tasks executed on 2 GPUs and 3 GPUs, respectively. Besides, the energy-saving profit displays a strong positive correlation to the reallocated GPU quantity. For one fine-tuning task, reallocation and down-clocking can effectively save energy costs with the extra advantages of boosted resource utilization and prolonged service life. Consequently, more idle GPUs reallocated to fine-tuning tasks in a greedy manner can bring more significant benefits.

Other than the volume of free GPU resources, the default resource allocation scheme (blue-colored Step 2 in Fig. 2) also impacts the energy-saving potentials, as demonstrated in Fig. 6. Scenario (a) resembles the setting above, whereas the cluster in scenario (b) carries 10, 20, and 10 tasks running on 1, 2, and 3 GPUs, respectively. As the energy-saving capability is more obvious when the GPU quantity for a task before reallocation is larger in this simulation study (indicated by Fig. 4), the cluster with more small tasks has a lower energy-saving potential. Moreover, the marginal energy-saving effects in the whole fine-tune cluster gradually drop with the increasing allocated GPU amount, because idle GPUs are always first reallocated to fine-tuning tasks with maximum energy-saving efficiency.

6.3. Cost-saving profits from multi-interval load flexibility

Fig. 7 illustrates how the power and performance of the GPU cluster changes with feasible core frequencies. The 'frequency'-load characteristic is illustrated in Fig. 7(a), which uncovers an enlarged adjustable power range in the upward trend with the increasing reallocated GPU quantity. This enlarged range is driven by more down-clocking GPUs in the expanded cluster that can increase power consumption by accelerating core frequency. Starting from the initial states, the power can be specifically tuned at least between -45% and 37% for the cluster with 30 GPUs reallocated to. Moreover, the accompanied impact of changing GPU frequency on the computing performance reflected in Fig. 7(b) indicates an increasing slope of power versus performance. To maintain

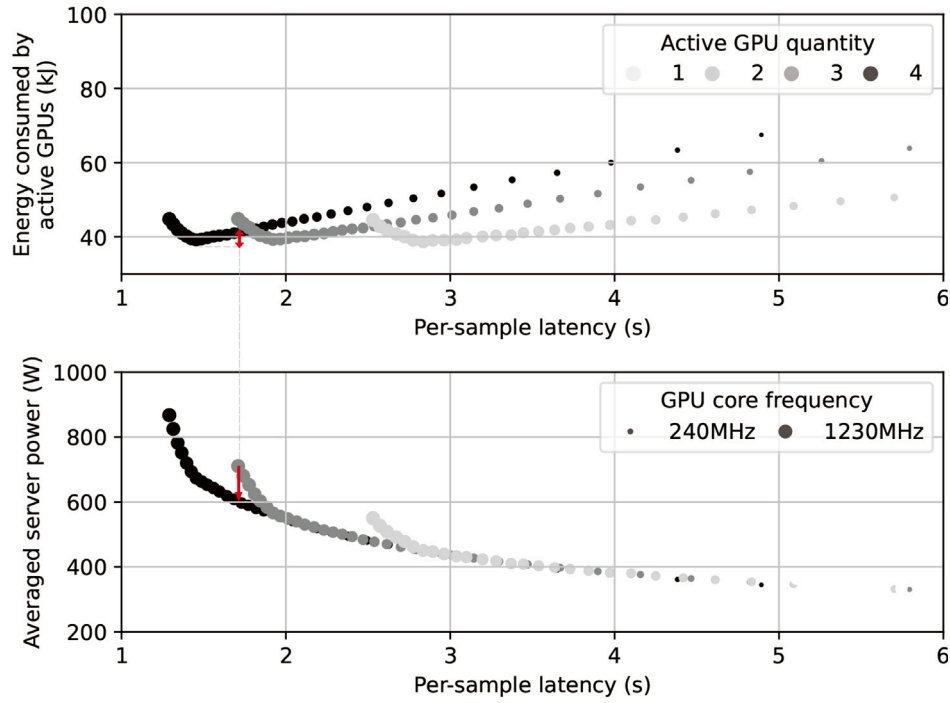


Fig. 4. Impacts of the GPU quantity and GPU core frequency on power and performance for LLM fine-tuning workload from real-world profiling results in a physical server with 4 NVIDIA A800 GPUs. Computing performance, represented by per-sample latency, remains unchanged when achieving energy-saving potentials shown as the red-colored one-way arrow.

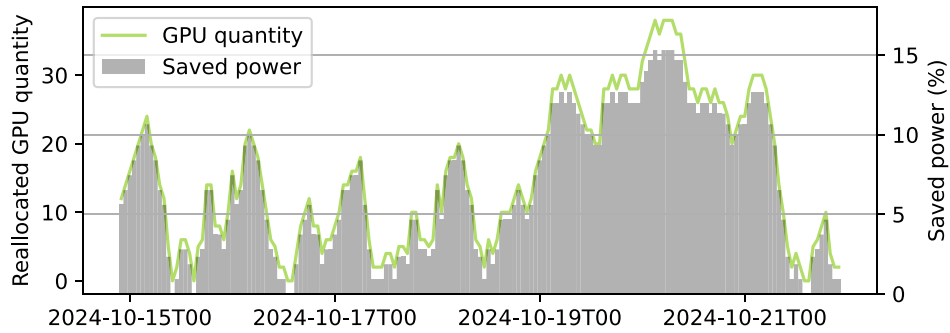


Fig. 5. Temporal energy-saving profits from reallocation and down-clocking obtained by comparing DASF against SASF.

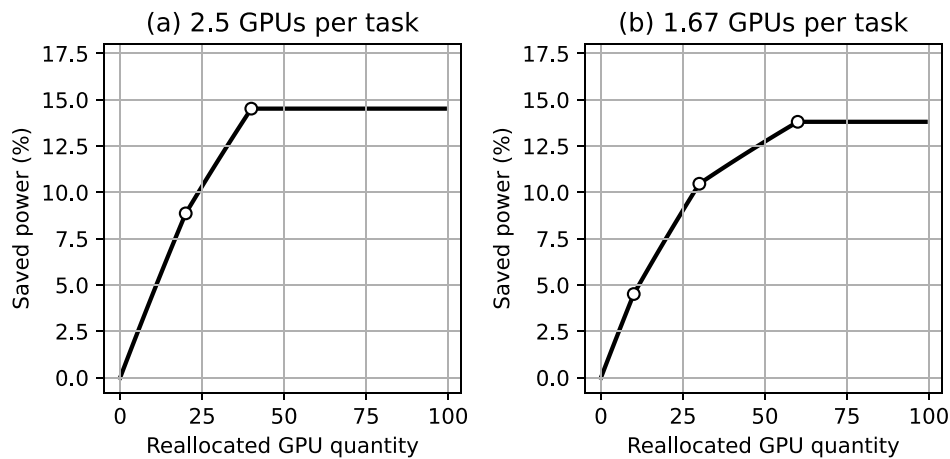


Fig. 6. Energy-saving profits under different default task-level allocation scenarios.

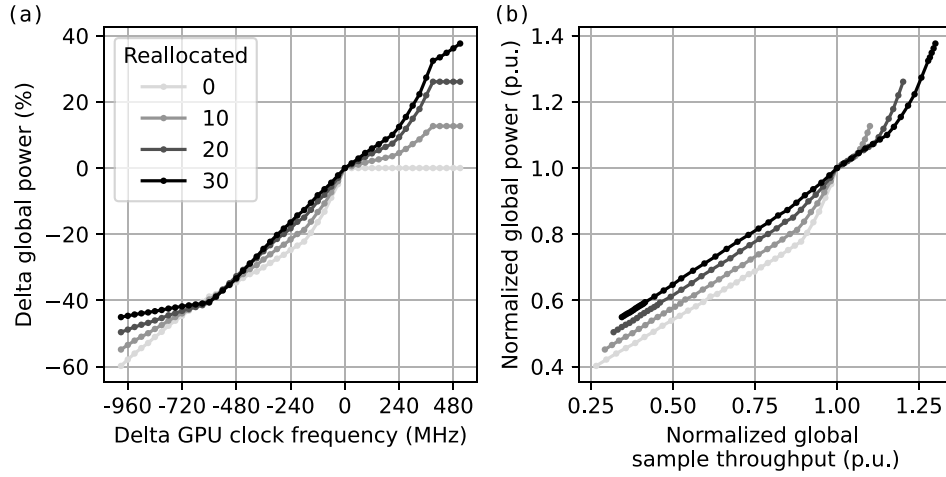


Fig. 7. 'Frequency'-load characteristics of the GPU cluster with varying reallocated GPU quantity, plus the accompanied impact of changing GPU frequency on the computing performance.

Table 1

Saved costs under pricing policies with different temporal resolution.

Pricing interval	1 h	5 min
Average	11.3%	13.2%
Maximum	24.0%	36.1%

the computing performance (which means the average of normalized global sample throughput must equal one) in multi-interval scenarios, the averaged power will exceed the base value if the power profile does not keep completely stable. In other words, the extracted multi-interval load flexibility is more suitable for the cases where volatility brings extra costs besides the total energy volume.

A significant cost-saving gain is witnessed by exploiting multi-interval load flexibility, as recorded in the top sub-figure of Fig. 8. Specifically, the comparison in electricity costs happens between the DADF and the SASF baseline. Overall, the expanded cluster can reduce and adjust the power profiles of the fine-tuning workload, and thus counteract the volatility of both renewable energy and inference requests. Contrarily, the default cluster cannot offer any flexibility without weakening computing performance, which is also reflected in Fig. 7(b). Besides, there is a negative correlation between the averaged net power of the inference workload (minus renewable generation) within one hour and the hourly cost-saving profit. The higher the net power of the inference workload is, the more volatility is incorporated in total power profiles, and the fewer idle computing resources there are. Both trends undermine the effects of scheduling fine-tuning workloads.

The load flexibility of fine-tuning tasks reduces overall power costs through two complementary mechanisms, as shown in the lower panels of Fig. 8. First, power consumption is reduced directly through GPU reallocation combined with down-clocking, mainly in response to volumetric tariffs. Second, peak power demand is redistributed across adjacent intervals, effectively mitigating capacity-based charges. As a result, the net power profile of the entire cluster is lower and smoother.

Crucially, the cost-saving effect is amplified when scheduling windows include multiple time-of-use tariff levels. To demonstrate the system's responsiveness to more dynamic signals, we simulate a pricing regime with volumetric tariffs at 5 min resolution (versus standard hourly adjustments), revealing a more significant cost reduction shown in Table 1. The maximum profit can even achieve an increase of 12.1% as compared to the original pricing scheme, indicating a stronger potential. This validates that pricing architectures exploiting sub-hourly granularity could better exploit the load flexibility of the LLM workload.

7. Discussion about future applications

Applicability under varying energy supply conditions - The proposed framework, validated in environments with high penetration of intermittent renewables, also holds potential applicability in settings with significant baseload power from sources such as nuclear and hydroelectricity. In such scenarios, while the economic incentives for demand-side flexibility are moderated, the framework's utility remains.

The contribution of single-interval flexibility lies in its direct impact on power reduction. Otherwise, multi-interval flexibility is necessitated by the inherent power dynamics of computational inference tasks. These intrinsic fluctuations in demand underscore the importance of dynamic power management for addressing two key considerations. First, by smoothing the aggregate power profile, the framework mitigates steep demand ramps. This reduces the need for infrastructure to be provisioned for peak loads that are infrequently reached. Second, data centers typically operate with a mixed-energy portfolio, supplementing baseload power contracts with grid electricity or on-site renewable generation. In these environments, the framework's ability to align computational demand with power availability is instrumental in maximizing the use of lower-cost energy.

Generalization to diverse workloads - The principles underpinning our framework are fundamentally applicable to a wider range of computational tasks. These workloads can be broadly classified into two archetypes. The first, represented by LLM inference, includes latency-sensitive, high-concurrency applications like live video streaming and real-time transcoding, which are characterized by bursty demand patterns. The second, exemplified by LLM fine-tuning, encompasses throughput-oriented batch workloads such as large-scale scientific simulations, where maximizing computational throughput is the primary goal. Our approach is well-suited to manage the distinct power and performance needs in environments that co-locate these diverse tasks, but with necessary customization.

One specific aspect of generalization is scalability, particularly for training large-scale foundation models. Our current work empirically validates the approach for fine-tuning workloads by end-to-end measurements. Extending the framework to foundation model training that spans hundreds or thousands of GPUs for merely one task introduces a new set of challenges. In such large-scale distributed environments, communication over networks like InfiniBand or Ethernet can become a dominant performance bottleneck. A more sophisticated power-performance model would be required for this scenario, one that explicitly accounts for network topology, the efficiency of collective communication patterns like All-Reduce, and the heterogeneity between fast intra-node and slower inter-node links. While our core

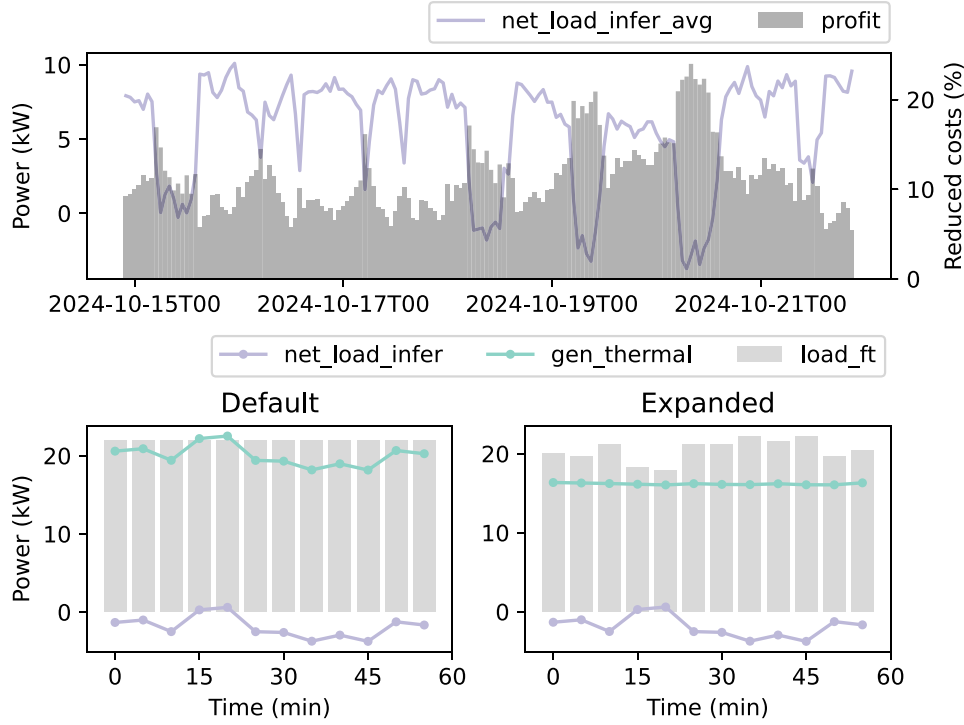


Fig. 8. Cost-saving profits from GPU frequency scaling after GPU reallocation, obtained by comparing DADF against SASF.

methodology remains relevant, adapting it to these large-scale systems by incorporating communication dynamics would be an important extension of this work.

Robustness to renewable and inference uncertainties - Our current study assumes accurate forecasts for inference demand and renewable generation to isolate and validate the effectiveness of the extracted load flexibility. Forecast errors hardly impact the computing performance, while they could affect the magnitude of realized energy and cost savings. Taking the multi-interval scenario as an example, the computing performance is specifically guaranteed by an independent constraint, but the optimal cost-saving profit may not be attained, as the parameters of the optimization model deviate from real values. To bolster the resilience to such inaccuracies, stochastic programming techniques could be introduced to optimize decisions under uncertainty.

8. Conclusion

Modern power systems require more flexible demand under future 100% renewable scenarios. Meanwhile, GPU data centers, emerging as energy monsters, demand a green-powered solution. An intriguing phenomenon that more GPUs and lower core frequency save power sources is first discovered and proved. Inspired by this, the GPU amount and GPU core frequency are taken as task-level control variables. This paper makes the first attempt to explore a possible way of flexibly adjusting the power consumption of compute-intense LLM tasks without computing performance degradation. For one, the expanded cluster after GPU reallocation can offer 6.8% power-saving capability on average with static frequency configuration, which ability highly depends on idle resources and can be utilized to participate in demand response programs. For another, the cluster can smartly follow renewable energy generation under dynamic frequency configuration. Consequently, GPU data centers can decrease electricity costs by 11.3% and accomplish social decarbonization obligations, and the power grid can be partly relieved from the pressure of accommodating stochastic volatile renewable generation.

In addition to the current validated scenarios, the proposed framework can be extended to other practical environments. Potential directions for future work include scalability to LLM training workloads and resilience to renewable and inference uncertainties.

CRediT authorship contribution statement

Yi Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Conceptualization. **Qinglai Guo:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Min Chen:** Writing – review & editing, Visualization, Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is sponsored by the National Natural Science Foundation of China (No. 52321004), and Tsinghua-Toyota Joint Research Institute Inter-disciplinary Program. The computing resources are supported by High Performance Computing Center, Tsinghua University.

Appendix A. Derivation details of propositions

A.1. Derivation of Proposition 1

Proof. The first-order derivative of $\hat{E}^n(f)$ can be written as $D(f) = \alpha f^2 + \beta f + \gamma - \theta f^{-2}$ where the coefficients $\alpha, \beta, \gamma, \theta$ satisfy $\alpha = 3\hat{\alpha}_j^T \alpha_j^P$, $\beta = 2\beta_j^T \alpha_j^P$, $\gamma = \hat{\alpha}_j^T \beta_j^P$, $\theta = \beta_j^T \gamma_j^P$. Given $D(\mu_1) = \sqrt{\frac{\gamma}{\theta}} = \alpha \frac{\gamma}{\theta} + \beta \sqrt{\frac{\gamma}{\theta}} > 0$, $D(\mu_2) = \min\{\sqrt{\frac{\theta}{4\alpha}}, \sqrt{\frac{\theta}{4\beta}}, \sqrt{\frac{\theta}{4\gamma}}\} \leq -\frac{\theta}{4\mu_2^2} < 0$ and $D(f)$ is strongly

increasing, there exists only one point $\xi \in (\mu_1, \mu_2)$ such that $D(\xi) = 0$. As the second-order derivative of $\hat{E}_j^n(\xi) = 2\alpha\xi^2 + \beta + 2\theta\xi^{-3}$ is positive, ξ is thus the minimum point of $\hat{E}_j^n(f)$. $\square$

A.2. Derivation of Proposition 2

Proof. Given the computing performance and the constant local batch size, configurations of these two clusters should satisfy (A.1). Combining (A.1) with (6) can derive n_j^1 as in (A.2).

The power comparison between two clusters is firstly rewritten as (A.3a) according to (A.1), where $n_j^q P_j(f_j^q)$ is the power consumed by Cluster- q . In order to utilize the energy feature for a fixed GPU amount uncovered in Proposition 1, an auxiliary term $T_j(n_j^0, f_j^1)$ is then introduced in (A.3b) to transform the comparison between configurations (n_j^0, f_j^0) and (n_j^1, f_j^1) into that between (n_j^0, f_j^0) and (n_j^0, f_j^1) . This transformation also separates the impact of core frequency f on power from that of GPU quantity n on the latency. Denote the temporal extension scale $T_j(n_j^1, f_j^1)/T_j(n_j^0, f_j^1)$ as κ due to prolonged communication latency. Hence, Cluster-1 consuming less power equals $\kappa < e_j^0/e_j^1 = E^{-1}$ as (A.3c) indicates.

Combining the definition of κ , the definition of T_j in (6) and the derivation of n_j^1 in (A.2) finally gives the equivalent inequation in (A.4).

$$\frac{T_j(n_j^1, f_j^1)}{n_j^1} = \frac{T_j(n_j^0, f_j^0)}{n_j^0} \quad (\text{A.1})$$

$$n_j^1 = \frac{\alpha_j^T + \beta_j^T / f_j^1}{\alpha_j^T + \beta_j^T / f_j^0} n_j^0 = \kappa n_j^0 \quad (\text{A.2})$$

$$\frac{n_j^1 P_j(f_j^1)}{n_j^0 P_j(f_j^0)} = \frac{T_j(n_j^1, f_j^1) P_j(f_j^1)}{T_j(n_j^0, f_j^0) P_j(f_j^0)} \quad (\text{A.3a})$$

$$= \frac{T_j(n_j^1, f_j^1) T_j(n_j^0, f_j^1) P_j(f_j^1)}{T_j(n_j^0, f_j^1) T_j(n_j^0, f_j^0) P_j(f_j^0)} \quad (\text{A.3b})$$

$$= \kappa \frac{\hat{T}_j^{n_j^0}(f_j^1) P_j(f_j^1)}{\hat{T}_j^{n_j^0}(f_j^0) P_j(f_j^0)} = \kappa \frac{e_j^1}{e_j^0} < 1 \quad (\text{A.3c})$$

$$\gamma_j^T (KE - 1) n_j^0 < (1 - E)(\alpha_j^T + \beta_j^T / f_j^1) \quad \square \quad (\text{A.4})$$

Appendix B. Profiling and modeling for inference

B.1. Basic models

The average power and completion time for an inference job on one GPU depend on the local batch size b and GPU core frequency f , which both have discrete setting values. Given the limited setting combinations (b, f) and fast inference execution, the empirical energy model $e(f, b)$ and performance model $\iota(f, b)$ can be easily obtained in the offline period, using merely one GPU. This offline experiment can reveal the real impacts of GPU configuration on power and time consumption without wasting GPU resources. Otherwise, building mathematical models will unavoidably introduce nonlinear relations and potentially non-convex optimization problems.

Then, the overall power and performance models for one inference workload with l requests are established by further analyzing the impact of the number of GPUs n . For simplicity, homogeneous computing resources are assumed here, where GPUs operate equally under the same configuration.

$$T(n, f, b, l) = \quad (\text{B.1})$$

$$\begin{cases} \iota(f, b) * \left(\left\lfloor \frac{l}{bn} \right\rfloor + 1 \right), l - \left\lfloor \frac{l}{bn} \right\rfloor * nb \geq b \\ \iota(f, b) * \left\lfloor \frac{l}{bn} \right\rfloor + \iota(f, l - \left\lfloor \frac{l}{bn} \right\rfloor * nb), \text{otherwise} \end{cases}$$

$$E(n, f, b, l) = e(f, b) * \left\lfloor \frac{l}{b} \right\rfloor + e(f, l - \left\lfloor \frac{l}{b} \right\rfloor b) \quad (\text{B.2})$$

where $\lfloor \cdot \rfloor$ denotes the floor function. Notably, we do not directly obtain the impact of n similar to that of (b, f) considering the possibilities of n are much more than b or f . Besides, the direct extension will occupy large amounts of GPUs and fail to promptly launch new inference tasks.

B.2. Configuration optimization

Primarily, LLM inference should be finished within the maximum latency threshold T_0 to satisfy consumer demands, regardless of GPU configuration and the request amount. This latency requirement is guaranteed by considering the possible peak request demand. Using the method in [38], we can predict the q -quantile of the inference workload $l_1^q, \dots, l_{N_t}^q$ in the next reallocation period. The sampling interval of the series is T_s . Thus, the maximum request amount L_m can be approximated as:

$$L_m = \frac{T_s}{T_0} \max_{t \in [N_t]} L_t^q \quad (\text{B.3})$$

On the premise of the latency requirement, the optimal number of GPUs n , GPU core frequency f , and local batch size b for one GPU can be determined by solving the following optimization problem, which minimizes total power consumption but ensures prompt response to consumers.

$$\min_{n, f, b} E(n, f, b, L_m) \quad (\text{B.4a})$$

$$\text{s.t. } T(n, f, b, L_m) \leq T_0 \quad (\text{B.4b})$$

B.3. Profiling results

Inference throughput increases with a larger batch size per GPU, but the marginal gain gradually drops, as demonstrated in Fig. B.9. It is evident that GPUs trade off latency for throughput using parallel computing techniques, and a larger batch normally represents more parallelism. The changing pattern of performance goes similarly regarding the GPU core frequency, where the descending marginal profit can be driven by memory operators included in the inference workload, which are insensitive to computing speed.

Moreover, GPU power consumption increases with higher clock frequencies and batch sizes. Power is roughly proportional to the cube of core frequency, and batch size affects power subtly due to slightly varying activity factors of GPU sub-modules. Overall, larger batch sizes improve time and energy efficiency, while moderate GPU frequencies optimize energy efficiency. The optimal batch size for minimizing energy consumption can also be seen in the upper-right-located subfigure of Fig. B.9. As the required response threshold decreases, the optimal batch size also drops due to better energy efficiency but longer latency for a large batch size configuration.

Data availability

Data will be made available on request.

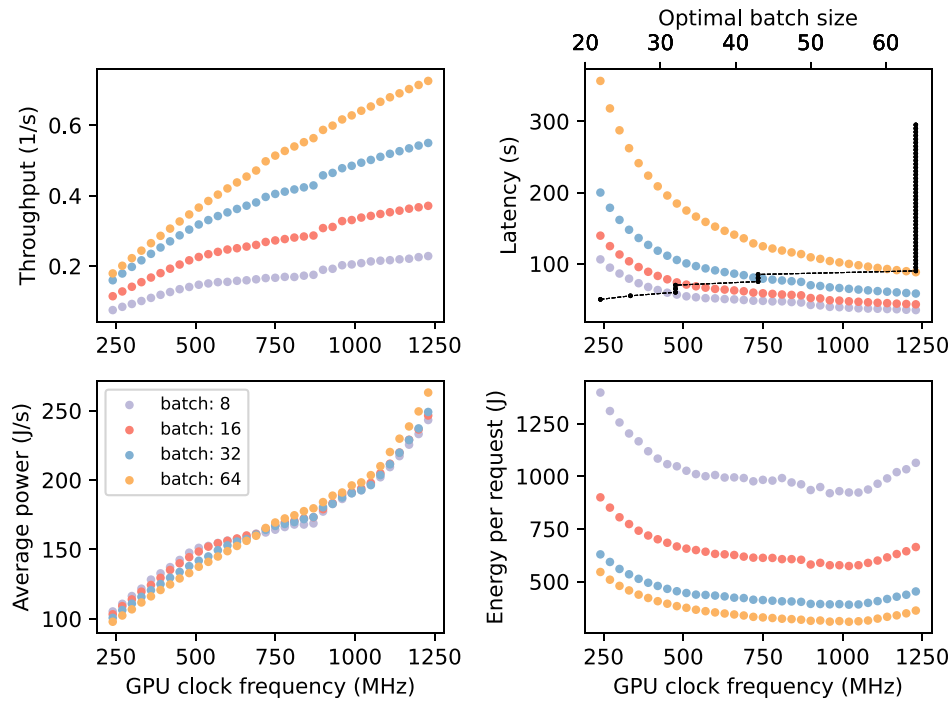


Fig. B.9. Profiling results of LLM inference workload.

References

- [1] Crawford K. Generative AI's environmental costs are soaring — and mostly secret. *Nature* 2024;626(8000). <http://dx.doi.org/10.1038/d41586-024-00478-x>, 693–693.
- [2] AI@Meta. Llama 3 model card. 2024, URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- [3] de Vries A. The growing energy footprint of artificial intelligence. *Joule* 2023;7(10):2191–4. <http://dx.doi.org/10.1016/j.joule.2023.09.004>.
- [4] AI is poised to drive 160% increase in data center power demand URL <https://www.goldmansachs.com/insights/articles/AI-poised-to-drive-160-increase-in-power-demand>.
- [5] Citi Research. Data center powerplay: The chips have to go somewhere URL <https://www.citigroup.com/global/insights/data-center-powerplay-the-chips-have-to-go-somewhere>.
- [6] Masanet E, Lei N, Koomey J. To better understand AI's growing energy use, analysts need a data revolution. *Joule* 2024;8(9):2427–36. <http://dx.doi.org/10.1016/j.joule.2024.07.018>.
- [7] EirGrid and SONI. Ireland Capacity Outlook 2022–2031 URL https://cms.eirgrid.ie/sites/default/files/publications/EirGrid_SONI_Ireland_Capacity_Outlook_2022-2031.pdf.
- [8] Datascares – An illustrated cartography of data center development in Quebec – VRM URL <https://www.vrm.ca/en/en-paysages-de-la-donnee-cartographie-illustree-du-developpement-des-centres-de-donnees-au-quebec/>.
- [9] Innovative data centres in Norway URL <https://www.polaris.com/data-centers>.
- [10] Sun H, Guo Q, Shen X, Xue Y, Shahidehpour M, Hatziaargyriou N. Energy internet: Redefinition and categories. *Energy Internet* 2024;1(1):3–8. <http://dx.doi.org/10.1049/ein2.12008>, URL <https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/ein2.12008>.
- [11] Riepin I, Brown T, Zavala VM. Spatio-temporal load shifting for truly clean computing. *Adv Appl Energy* 2025;17:100202. <http://dx.doi.org/10.1016/j.adapen.2024.100202>, URL <https://www.sciencedirect.com/science/article/pii/S2666792424000404>.
- [12] IEA. Top corporate off-takers of renewable energy power purchase agreements, 2010–2022. 2023, <https://www.iea.org/data-and-statistics/charts/top-corporate-off-takers-of-renewable-energy-power-purchase-agreements-2010-2022>.
- [13] Guerreiro J, Ilic A, Roma N, Tomás P. DVFS-aware application classification to improve GPGPU energy efficiency. *Parallel Comput* 2019;83:93–117. <http://dx.doi.org/10.1016/j.parco.2018.02.001>, URL <https://www.sciencedirect.com/science/article/pii/S0167819118300243>.
- [14] Palensky P, Dietrich D. Demand side management: Demand response, intelligent energy systems, and smart loads. *IEEE Trans Ind Informatics* 2011;7(3):381–8. <http://dx.doi.org/10.1109/TII.2011.2158841>.
- [15] Chen M, Gao C, Shahidehpour M, Li Z. Incentive-compatible demand response for spatially coupled internet data centers in electricity markets. *IEEE Trans Smart Grid* 2021;12(4):3056–69. <http://dx.doi.org/10.1109/TSG.2021.3053433>.
- [16] Chen X. Enhance low-carbon power system operation via carbon-aware demand response. *Energy Internet* 2024;1(2):141–9. <http://dx.doi.org/10.1049/ein2.12004>.
- [17] Li C, Qouneh A, Li T. Iswitch: Coordinating and optimizing renewable energy powered server clusters. In: 2012 39th annual international symposium on computer architecture. ISCA, 2012, p. 512–23. <http://dx.doi.org/10.1109/ISCA.2012.6237044>.
- [18] Ye Z, Gao W, Hu Q, Sun P, Wang X, Luo Y, Zhang T, Wen Y. Deep learning workload scheduling in GPU datacenters: A survey. *ACM Comput Surv* 2024;56(6):146:1–38. <http://dx.doi.org/10.1145/3638757>.
- [19] You J, Chung J-W, Chowdhury M. Zeus: Understanding and optimizing GPU energy consumption of DNN training. 2022, <http://dx.doi.org/10.48550/arXiv.2208.06102>, arXiv:2208.06102.
- [20] Park H-G, Kang D-K. Renewable-aware frequency scaling approach for energy-efficient deep learning clusters. *Appl Sci* 2024;14(2):776. <http://dx.doi.org/10.3390/app14020776>.
- [21] Gu D, Xie X, Huang G, Jin X, Liu X. Energy-efficient GPU clusters scheduling for deep learning. 2023, arXiv:2304.06381.
- [22] Yao C, Liu W, Tang W, Hu S. EAIS: Energy-aware adaptive scheduling for CNN inference on high-performance GPUs. *Future Gener Comput Syst* 2022;130:253–68. <http://dx.doi.org/10.1016/j.future.2022.01.004>.
- [23] Nabavinejad SM, Guo T. Opportunities of renewable energy powered DNN inference. 2023, arXiv:2306.12247.
- [24] Kandiah V, Peverelle S, Khairi M, Pan J, Manjunath A, Rogers TG, Aamodt TM, Hardavellas N. Accelwatch: A power modeling framework for modern GPUs. In: MICRO-54: 54th annual IEEE/ACM international symposium on microarchitecture. MICRO '21, New York, NY, USA: Association for Computing Machinery; 2021, p. 738–53. <http://dx.doi.org/10.1145/3466752.3480063>.
- [25] Butts J, Sohi G. A static power model for architects. In: Proceedings 33rd annual IEEE/ACM international symposium on microarchitecture. MICRO-33 2000. 2000, p. 191–201. <http://dx.doi.org/10.1109/MICRO.2000.898070>.
- [26] Gonzalez R, Gordon B, Horowitz M. Supply and threshold voltage scaling for low power CMOS. *IEEE J Solid-State Circuits* 1997;32(8):1210–6. <http://dx.doi.org/10.1109/4.604077>.
- [27] Jain S, Khare S, Yada S, Ambili V, Salihundam P, Ramani S, Muthukumar S, Srinivasan M, Kumar A, Gb SK, Ramanarayanan R, Erraguntla V, Howard J, Vangal S, Dighe S, Ruhl G, Aseron P, Wilson H, Borkar N, De V, Borkar S. A 280mv-to-1.2v wide-operating-range IA-32 processor in 32nm CMOS. In: 2012 IEEE international solid-state circuits conference. 2012, p. 66–8. <http://dx.doi.org/10.1109/ISSCC.2012.6176932>.
- [28] Zimmer B, Lee Y, Puggelli A, Kwak J, Jevtic R, Keller B, Bailey S, Blagojevic M, Chiu P-F, Le H-P, Chen P-H, Sutardja N, Avizienis R, Waterman A, Richards B, Flatresse P, Alon E, Asanović K, Nikolić B. A RISC-v vector processor with tightly-integrated switched-capacitor DC-dc converters in 28nm FDSOI. In: 2015 symposium on VLSI circuits (VLSI circuits). 2015, p. C316–7. <http://dx.doi.org/10.1109/VLSIC.2015.7231305>.

- [29] Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In: 15th USENIX symposium on operating systems design and implementation (OSDI 21). USENIX Association; 2021, p. 1–18, URL <https://www.usenix.org/conference/osdi21/presentation/qiao>.
- [30] Corporation N. NVIDIA L40s product deck. 2023, [Accessed 30 January 2025] <https://weareluniq.com/wp-content/uploads/2023/11/L40S-Product-Deck.pdf>.
- [31] Stojkovic J, Zhang C, Goiri I, Torrellas J, Choukse E. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. 2024, <http://dx.doi.org/10.48550/arXiv.2408.00741>, arXiv:2408.00741.
- [32] Corporation N. NVIDIA management library (NVML) URL <https://developer.nvidia.com/management-library-nvml>.
- [33] Velicka D, Vysocky O, Riha L. Methodology for GPU frequency switching latency measurement. 2025, <http://dx.doi.org/10.48550/ARXIV.2502.20075>, URL <https://arxiv.org/abs/2502.20075> Version Number: 1.
- [34] Qwen Team. Qwen2.5: A party of foundation models. 2024, URL <https://qwenlm.github.io/blog/qwen2.5/>.
- [35] Leswing K. Meet the \$10,000 nvidia chip powering the race for a.i.. 2023, URL <https://www.cnbc.com/2023/02/23/nvidias-a100-is-the-10000-chip-powering-the-race-for-ai-.html> Section: Technology,
- [36] West S. ARENA solar power ensemble forecaster project - solar farm final dataset. 2020, <http://dx.doi.org/10.25919/y25q-wt34>, URL <https://doi.org/10.25919/y25q-wt34> Publisher: CSIRO.
- [37] Draxl C, Clifton A, Hodge B-M, McCaa J. The wind integration national dataset (WIND) toolkit. Appl Energy 2015;151:355–66. <http://dx.doi.org/10.1016/j.apenergy.2015.03.121>.
- [38] Salinas D, Flunkert V, Gasthaus J, Januschowski T. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. Int J Forecast 2020;36(3):1181–91. <http://dx.doi.org/10.1016/j.ijforecast.2019.07.001>.